Gateway Guard

Product note and connection architecture

1. What it is

Gateway Guard is the desktop agent that sits on the employee PC.
It watches AI websites the company has marked as targets
(ChatGPT, Claude, Gemini, DeepSeek, and the rest of the target list).
A prompt, a file, or a voice message is checked before it leaves the browser.
The result is Allowed, Warn, Redact, or Block, and the event is stored
as a Prompt Log on the Gateway.

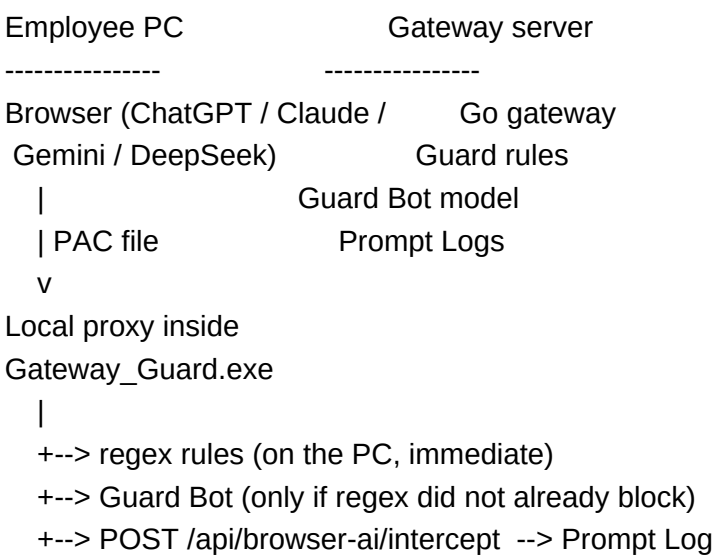
2. Languages

Desktop agent: Python 3, packaged with PyInstaller into Gateway_Guard.exe.
Local proxy: mitmproxy, started by that same exe.
Windows installer: Inno Setup 6. Setup file is Gateway_Guard_Setup.exe.
Current installer version string: 1.1.23.
Gateway server: Go. Admin screens: TypeScript, React, and Vite.
The Go process serves the admin UI and the Browser AI APIs.

3. What the setup contains

Install folder: %LOCALAPPDATA%\Programs\Gateway\Guard
Gateway_Guard.exe is the program the user runs. It can start at login.
gateway_guard_config.json is the server address and guard settings.
A fresh copy next to the setup file replaces the one baked into the build.
Encrypted bundles (not readable source):
  gateway_guard_code.enc
  gateway_proxy_parts.enc
The proxy loads the encrypted parts in this order:
  config_caches_rules, helpers_prompts, uploads_detect,
  file_policy, extract_office_backend,
  responses_inject, responses_addon.
proxy.pac tells the browser which sites go through the local proxy.
The mitmproxy certificate is installed so HTTPS on those sites can be read.

4. Connection structure



Step by step

1. The employee opens an AI site in the browser.
2. The PAC file sends that site to the local proxy. Other sites go direct.
3. The user types a prompt, attaches a file, or sends voice, then presses Send.
4. The proxy takes the text. A file is read from the recent upload cache.
   Voice is the transcribed text. Upload alone does not run a decision.
5. Local regex rules run first. A regex BLOCK stops there. It does not wait
   for the Guard Bot.
6. If regex did not block, the Guard Bot on the Gateway can still decide.
   The stricter result wins.

5. The four results

Allowed The prompt goes on to the AI site.
Warn The prompt goes on, and the event is marked as a warning.
Redact The matched text is replaced with [REDACTED], then sent on.
Block The send stops. The chat page shows the block notice.
The proxy writes that notice into the site response.

Every enforced result is posted to the Gateway and appears in Prompt Logs
with the site, the prompt, the action, and the reason.

6. What each proxy part does

config_caches_rules Target sites, guard rules, and the Allow/Warn/Redact/Block decision.
helpers_prompts Pulls the prompt text out of each site's request.
uploads_detect Remembers a file the user attached, until Send.
file_policy Size and type limits for that file.
extract_office_backend Reads text from Office and PDF files.
responses_inject Writes the block notice into the AI site's reply.
responses_addon mitmproxy hook that ties the parts to each request.

7. Rule example

In Create Guard Rule, a short description is turned into one regex.
The saved rule is that regex, not the description.
A description of a secret (pin code, email, phone, card) must match the
secret itself. Pin code is a 4 to 6 digit number, pattern \b\d{4,6}\b.
It does not match the words pin or code.
Severity and action (BLOCK, REDACT, WARN) are chosen on the same form.
The test box shows the result before the rule is saved.

8. What stays on the PC and what stays on the server

On the PC: the exe, the local proxy, the PAC file, the certificate,
and a cache of the rules so a regex block is immediate.
On the server: the rule list, the Guard Bot, virtual keys, and Prompt Logs.
The PC calls the server for rules, for the bot decision, and to store the log.